

Update

```
$affected = DB::update('update users set votes = 100 where name = ?', ['Paul']);
```

Delete

```
$deleted = DB::delete('delete from users');
```

Sample general statement

```
DB::statement('drop table users');
```

Query builder

With Laravel Query builder, you can easily create and run database queries. The builder makes use of the PDO parameter, which will protect your application against SQL injection attacks without any need to flush out strings.

To fetch all rows from specific table, you can use the table method on the DB facade via `get()` method. If you would like to retrieve a single row, you should use `first()` method without any parameters, which returns a single `stdClass` object. You can also use `value()` method with a specific table name as parameter instead of `first()` to fetch a single value from a record. The `pluck()` method with the relevant parameter can be used in case if you want to fetch a collection list of column values.

You can easily retrieve large number of database records using the `chunk()` method by specifying the total number of records to be recovered. Sometimes, you will be required to list the values from the table based on specific criteria. In such a case, you can use aggregate methods such as `count`, `max`, `min`, `avg`, and `sum`. For instance, if you provide the query `$price = DB::table('orders')->max('price');` inside the builder, you will be able to view the details of the orders sorted by price. The `exists()` and `doesntExist()` methods can be used to determine whether the record that matches with the query statements exists.

The `select()` method can be used to fetch a particular set of records from the database using a customized clause. If you apply `distinct()` method, you can instruct the query to only return distinct results. The raw expression can be created using the `DB::raw` method. It is also possible to plug a raw expression with the help of `selectRaw`, `whereRaw`, or `WhereRaw`, `havingRaw`, or `HavingRaw` and `orderByRaw` methods.

With Laravel query builder, you can easily write join statements by using the `join()`, `leftJoin()`, `crossJoin()` methods including the